

强化学习与决策智能基础

强化学习是连接 " 理解 " 与 " 行动 " 的关键桥梁——它让机器学会做决策。

强化学习与决策智能基础

强化学习是连接 " 理解 " 与 " 行动 " 的关键桥梁——它让机器学会做决策。

- - -

一、核心思想：从 " 预测 " 到 " 决策 "

1 . 1 强化学习 v s 监督学习

维度 监督学习 强化学习

学习方式 从标注数据学习 从交互经验中学习

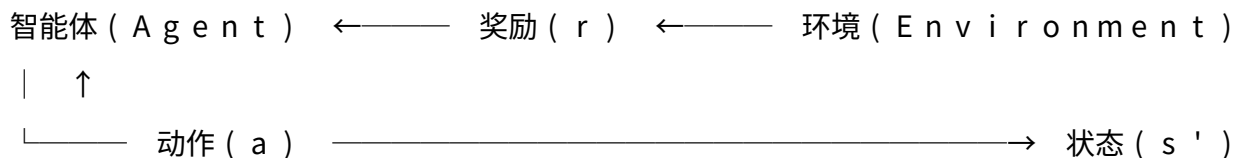
反馈信号 即时、密集（每个样本有标签） 延迟、稀疏（只有奖励）

目标 预测准确 长期累积奖励最大

数据 静态数据集 动态交互产生

核心挑战 过拟合 / 欠拟合 探索 - 利用困境

1 . 2 闭环交互过程



循环：观察状态 → 选择动作 → 获得奖励 → 观察新状态 → . . .

现实类比：

- 学骑自行车：你不会提前知道每个动作的 " 正确答案 "，只能通过摔跤（负奖励）和成功骑行（正奖励）来学习。
- 下棋：每步没有标签告诉你该怎么走，只有最后的胜负告诉你结果

- - -

二、马尔可夫决策过程 (M D P)

2 . 1 M D P 的严格定义

M D P 是强化学习问题的形式化框架，定义为五元组 S, A, P, R, γ ：

符号 含义 示例（自动驾驶）

S 状态集合 车辆位置、速度、周围车辆状态

A 动作集合 加速、刹车、转向

$P(s' | s, a)$ 状态转移概率 执行动作后到达新状态的概率

$R(s, a)$ 奖励函数 安全行驶 + 1，碰撞 - 1 0 0

$\gamma \in [0, 1]$ 折扣因子 未来奖励的重要性（ γ 接近 1 重视长期）

2.2 马尔可夫性质

马尔可夫性质：未来只与当前状态有关，与过去无关。

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} | s_t, a_t)$$

为什么重要？ 马尔可夫性质使得我们不需要记住完整的历史，只需当前状态就能做最优决策。

当状态不满足马尔可夫性时：

- 使用状态增强：将历史信息加入状态（如 L S T M 编码历史）
- 使用 P O M D P（部分可观察 M D P）框架

2.3 优化目标

最大化期望累计折扣奖励：

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

其中 $\pi(a | s)$ 是策略——在状态 s 下选择动作 a 的概率。

折扣因子的意义：

- $\gamma = 0$ ：完全短视，只关心即时奖励
- $\gamma = 1$ ：完全远视，远期奖励与即时奖励同等重要
- $\gamma = 0.99$ （常用）：稍微重视当前，但长期规划

- - -

三、价值函数与 B e l l m a n 方程

3.1 状态价值函数 $V(s)$

从状态 s 出发，遵循策略 π ，期望获得的累计折扣奖励：

$$V^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

直觉： $V(s)$ 告诉你 " 处于状态 s 有多好 "。

3.2 动作价值函数 $Q(s, a)$

从状态 s 执行动作 a 后，遵循策略 π ，期望获得的累计折扣奖励：

$$Q^\pi(s, a) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right]$$

直觉： $Q(s, a)$ 告诉你 " 在状态 s 执行动作 a 有多好 "。

3.3 两者关系

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

关键结论：如果我们能准确估计 $Q(s, a)$ ，最优策略就是：

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

这就是为什么 Q -learning 如此重要——它直接学习 Q 函数。

3.4 Bellman 方程详解

Bellman 期望方程（对任意策略 π ）：

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

直觉：当前状态的价值 = 即时奖励的期望 + 下一状态价值的折扣期望

Bellman 最优方程：

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

核心含义：当前的最优值 = 立即奖励 + 折扣后的未来最优值

- - -

四、动态规划方法

4.1 策略评估

给定策略 π ，计算其价值函数 V^π 。

```
def policyevaluation(env, policy, gamma=0.99, theta=1e-6):
    """ 策略评估：计算给定策略的价值函数 """
    V = np.zeros(env.nstates)
    while True:
        delta = 0
        for s in range(env.nstates):
            v = 0
            for a in range(env.nactions):
                for snext in range(env.nstates):
                    prob = env.P(s, a, snext)
                    reward = env.R(s, a)
                    v += policy[a, s] * prob * (reward + gamma * V[snext])
            delta = max(delta, abs(v - V[s]))
        V[s] = v
        if delta < theta:
            break
    return V
```

4.2 策略改进

给定价值函数 V ，找到更好的策略 π' 。

策略改进定理：如果 $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$ 对所有 s 成立，则 $\pi' \geq \pi$ 。

4.3 策略迭代

交替执行策略评估和策略改进，直到策略不再变化。

```
def policyiteration(env, gamma=0.99):
    policy = np.ones((env.nactions, env.nstates)) / env.nactions
    while True:
        # 策略评估
        V = policyevaluation(env, policy, gamma)
        # 策略改进
        polycystable = True
        for s in range(env.nstates):
            oldaction = np.argmax(policy[:, s])
            # 对每个动作计算Q值
            qvalues = np.zeros(env.nactions)
            for a in range(env.nactions):
                for snext in range(env.nstates):
                    qvalues[a] += env.P(s, a, snext) * (env.R(s, a) + gamma * V[snext])
            newaction = np.argmax(qvalues)
            if oldaction != newaction:
                polycystable = False
                policy[:, s] = 0
                policy[newaction, s] = 1
            if polycystable:
                break
        return policy, V
```

4.4 值迭代

直接利用 Bellman 最优方程迭代更新，不需要显式维护策略：

```
def valueiteration(env, gamma=0.99, theta=1e-6):
    V = np.zeros(env.nstates)
    while True:
        delta = 0
        for s in range(env.nstates):
            qvalues = []
```

```

for a in range(env.nactions):
    q = sum(env.P(s, a, snext) * (env.R(s, a) + gamma * V[snext]))
    for snext in range(env.nstates):
        qvalues.append(q)
    vnew = max(qvalues)
    delta = max(delta, abs(vnew - V[s]))
V[s] = vnew
if delta < theta:
    break
return V

- - -

```

五、Q-learning（无模型方法）

5.1 时序差分学习（TD Learning）

TD Learning 结合了动态规划的自举（bootstrapping）和蒙特卡洛的采样：

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

TD误差： $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

5.2 Q-learning 算法

Q-learning 是 off-policy 的 TD 控制方法，直接学习最优Q函数：

```

def qlearning(env, nepisodes=10000, alpha=0.1, gamma=0.9):
    Q = np.zeros((env.nstates, env.nactions))

    for episode in range(nepisodes):
        s = env.reset()
        done = False

        while not done:
            # ε-greedy 策略：探索 vs 利用
            if np.random.random() < epsilon:

```

```

a = env.action_space.sample() # 随机探索
else:
a = np.argmax(Q[s]) # 利用当前最优

# 执行动作
snext, r, done, info = env.step(a)

# Q值更新（核心公式）
#  $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$ 
tdtarget = r + gamma * np.max(Q[snext]) * (1 - done)
tderror = tdtarget - Q[s, a]
Q[s, a] += alpha * tderror

s = snext

return Q

```

Q-learning 的关键特点：

- Off-policy：学习的是最优策略的Q值，而行为策略是 ϵ -greedy
- 收敛性：在充分探索的条件下，Q-learning 保证收敛到最优Q函数

5.3 SARSA 算法

SARSA 是 on-policy 版本，使用实际采取的下一个动作更新Q值：

Q-learning: $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$

SARSA: $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot Q(s', a') - Q(s, a)]$

↑ 用实际的下一个动作 a' 而非 $\max$

区别：

Q-learning 学习最优策略（不关心行为策略是什么）

SARSA 学习当前行为策略（更保守，但更稳定）

5.4 探索策略

策略 原理 优点 缺点

ϵ -greedy ϵ 概率随机， $1 - \epsilon$ 概率选最优 简单 探索无方向性

U C B 选不确定度高的动作 高效探索 需要维护置信区间

T h o m p s o n S a m p l i n g 从后验分布采样 贝叶斯最优 计算成本

熵正则化 在目标函数中加入策略熵 自动探索 需要调温度系数

- - -

六、深度Q网络（DQN）

6.1 为什么需要DQN？

当状态空间巨大时（如A t a r i 游戏的 210×160 像素），无法用Q表存储所有Q值。DQN用神经网络

6.2 DQN 的两大关键技术

1. 经验回放（Experience Replay）：

将交互经验存入缓冲区，训练时随机采样，打破时间相关性。

```
from collections import deque
import random

class ReplayBuffer:
    def __init__(self, capacity=100000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, nextstate, done):
        self.buffer.append((state, action, reward, nextstate, done))

    def sample(self, batchsize):
        batch = random.sample(self.buffer, batchsize)
        states, actions, rewards, nextstates, dones = zip(*batch)
        return (np.array(states), np.array(actions), np.array(rewards),
                np.array(nextstates), np.array(dones))

    def len(self):
        return len(self.buffer)
```

2. 目标网络（Target Network）：

复制一个延迟更新的网络计算目标Q值，避免训练震荡。

```
class DQNAgent:
    def __init__(self, statedim, actiondim, lr=1e-3, gamma=0.99, epsilon=1.0, epsilonmin=0.01, epsilondecay=0.995):
        self.statedim = statedim
        self.actiondim = actiondim
        self.gamma = gamma
        self.epsilon = epsilon

    # 主网络（频繁更新）
    self.qnetwork = self.buildnetwork()

    # 目标网络（延迟更新）
    self.targetnetwork = self.buildnetwork()
    self.targetnetwork.load_state_dict(self.qnetwork.state_dict())

    self.optimizer = torch.optim.Adam(self.qnetwork.parameters())
    self.buffer = ReplayBuffer(100000)
    self.update_target_every = 1000
    self.stepcount = 0

    def buildnetwork(self):
        return nn.Sequential(
            nn.Linear(self.statedim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, self.actiondim)
        )

    def act(self, state):
        if np.random.random() < self.epsilon:
            return np.random.randint(self.actiondim)
        with torch.no_grad():
```

```

statet = torch.FloatTensor(state).unsqueeze(0)
qvalues = self.qnetwork(statet)
return qvalues.argmax().item()

def update(self, batchsize=64):
    if len(self.buffer) < batchsize:
        return

    states, actions, rewards, nextstates, done = self
    states = torch.FloatTensor(states)
    actions = torch.LongTensor(actions)
    rewards = torch.FloatTensor(rewards)
    nextstates = torch.FloatTensor(nextstates)
    done = torch.FloatTensor(done)

    # 当前Q值
    currentq = self.qnetwork(states).gather(1, actions)

    # 目标Q值 (用目标网络计算)
    with torch.no_grad():
        maxnextq = self.targetnetwork(nextstates).max(1)
        targetq = rewards + self.gamma maxnextq (1 - done)

    # 计算损失并更新
    loss = nn.MSELoss()(currentq, targetq)
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 定期更新目标网络
    self.stepcount += 1
    if self.stepcount % self.updatetargetevery == 0:
        self.targetnetwork.loadstatedict(self.qnetwork.state_dict())

    # 衰减  $\epsilon$ 
    self.epsilon = max(self.epsilonmin, self.epsilon - 0.001)

```

6.3 DQN 的改进

6.3.1 Double DQN

问题：标准DQN的Q值过高估计（overestimation）。因为 $\max$ 操作总是偏向选择被过

解决：将动作选择和动作评估解耦——用主网络选动作，用目标网络评估Q值：

标准DQN（过高估计）：

$td_{target} = r + \gamma \max_{a'} Q_{target}(s', a')$

↑ 选择和评估都用目标网络

Double DQN（减少过高估计）：

1. 主网络选择动作： $a = \arg\max_a Q_{main}(s', a)$

2. 目标网络评估Q值： $td_{target} = r + \gamma Q_{target}(s', a)$

↑ 选择用主网络，评估用目标网络

```
class DoubleDQNAgent(DQNAgent):
```

```
def update(self, batchsize=64):
```

```
if len(self.buffer) < batchsize:
```

```
return
```

```
states, actions, rewards, nextstates, dones = self
```

```
states = torch.FloatTensor(states)
```

```
actions = torch.LongTensor(actions)
```

```
rewards = torch.FloatTensor(rewards)
```

```
nextstates = torch.FloatTensor(nextstates)
```

```
dones = torch.FloatTensor(dones)
```

```
# 当前Q值
```

```
currentq = self.qnetwork(states).gather(1, actions)
```

```
# Double DQN：主网络选动作，目标网络评估
```

```
with torch.no_grad():
```

```
# 主网络选择最优动作
```

```
bestactions = self.qnetwork(nextstates).argmax(1)
```

目标网络评估该动作的Q值

```
nextq = self.targetnetwork(nextstates).gather(1,
targetq = rewards + self.gamma nextq (1 - done)

loss = nn.MSELoss()(currentq, targetq)
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
```

6.3.2 Dueling DQN

核心理想：将Q函数分解为状态价值V(s)和优势函数A(s, a)：

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$$

减去均值是为了可辨识度（否则V和A可以随意偏移）。

直觉：很多状态下选择哪个动作区别不大（V主导），只有少数关键状态需要精细选择（A主导）。Dueling网络更容易学习这种结构。

```
class DuelingDQN(nn.Module):
    def __init__(self, statedim, actiondim, hiddendim=128):
        super().__init__()
        # 共享特征提取层
        self.feature = nn.Sequential(
            nn.Linear(statedim, hiddendim),
            nn.ReLU()
        )
        # 状态价值流 V(s)
        self.valuestream = nn.Sequential(
            nn.Linear(hiddendim, hiddendim),
            nn.ReLU(),
            nn.Linear(hiddendim, 1) # 输出1个标量
        )
        # 优势流 A(s, a)
        self.advantagestream = nn.Sequential(
```

```

nn.Linear(hiddendim, hiddendim),
nn.ReLU(),
nn.Linear(hiddendim, actiondim) # 输出 A 个值
)

def forward(self, x):
    features = self.feature(x)
    value = self.valuestream(features) # (batch, 1)
    advantage = self.advantagestream(features) # (batch, 1)
    # Q = V + A - mean(A)
    q = value + advantage - advantage.mean(dim=1, keepdim=True)
    return q

```

6.3.3 Prioritized Experience Replay

核心思想：不是均匀随机采样经验，而是优先采样TD误差大的经验（这些经验有更多“可学习”的信息）。

```

import heapq

class PrioritizedReplayBuffer:
    """ 优先经验回放 """
    def __init__(self, capacity=100000, alpha=0.6, beta=0.4):
        self.capacity = capacity
        self.alpha = alpha # 优先级指数：0 = 均匀采样，1 = 完全按优先级
        self.beta = beta # 重要性采样指数：0 = 无修正，1 = 完全修正
        self.beta_increment = 1e-6
        self.buffer = []
        self.priorities = [] # 优先级 (TD误差的绝对值 + ε)
        self.pos = 0

    def push(self, state, action, reward, nextstate, tderror):
        """ 存入经验，优先级 = |TD误差| + ε """
        priority = (abs(tderror) + 1e-6) ** self.alpha

        if len(self.buffer) < self.capacity:
            self.buffer.append((state, action, reward, nextstate, priority))
            self.priorities.append(priority)
            self.pos += 1
        else:
            # 使用堆来维护优先级
            heapq.heappushpop(self.priorities, priority)
            # 找到被替换的元素的索引
            for i, p in enumerate(self.priorities):
                if p == priority:
                    self.priorities[i] = priority
                    self.buffer[i] = (state, action, reward, nextstate, priority)
                    break
            self.pos = self.pos - 1 + 1

```

```

self.priorities.append(priority)
else:
    self.buffer[self.pos] = (state, action, reward, nextstate)
    self.priorities[self.pos] = priority

self.pos = (self.pos + 1) % self.capacity

def sample(self, batchsize):
    """按优先级采样"""
    priorities = np.array(self.priorities)
    probs = priorities / priorities.sum()
    indices = np.random.choice(len(self.buffer), batchsize)

    # 重要性采样权重 (修正偏差)
    N = len(self.buffer)
    weights = (N * probs[indices]) * (-self.beta)
    weights = weights / weights.max() # 归一化

    # 更新beta (逐渐增大到1)
    self.beta = min(1.0, self.beta + self.betaincrement)

    batch = [self.buffer[i] for i in indices]
    states, actions, rewards, nextstates, dones = zip(*batch)

    return (np.array(states), np.array(actions), np.array(rewards),
            np.array(nextstates), np.array(dones), indices, weights)

def updatepriorities(self, indices, tderrors):
    """更新采样经验的优先级"""
    for idx, tderror in zip(indices, tderrors):
        self.priorities[idx] = (abs(tderror) + 1e-6) * self.beta

```

6.3.4 DQN改进总结

改进 原理 效果 实现难度

Double DQN 主网络选动作, 目标网络评估 减少Q值过高估计 低

Dueling DQN $Q = V + A - \text{mean}(A)$ 更好地学习状态价值 低
 Prioritized Replay 优先采样TD误差大的经验 学习更高效 中
 Noisy Net 在权重上加噪声代替 ϵ -greedy 更有方向的探索 中
 C51 学习Q值的分布而非期望 更丰富的信息 高
 Rainbow 集成6种改进 显著提升性能 高

Rainbow = Double DQN + Dueling + Prioritized + Noisy Net + C51 + TD0.25 + W-clip
 - - -

七、策略梯度方法

7.1 为什么需要策略梯度？

方法 适用场景 局限

值函数方法 离散动作空间 无法处理连续动作

策略梯度 连续+离散动作空间 方差大，训练不稳定

7.2 策略梯度定理

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi(a_t | s_t; \theta) (G_t - V(s_t)) \right]$$

直觉：

- 如果某个动作导致高回报 (G_t 大)，增加该动作的概率
- 如果某个动作导致低回报 (G_t 小)，降低该动作的概率

REINFORCE 算法*：

```
def reinforce(env, policy_network, n_episodes=1000,
              optimizer=torch.optim.Adam(policy_network.parameters())):
    for episode in range(n_episodes):
        # 收集一整个episode
        log_probs = []
        rewards = []
        state = env.reset()
```

```

done = False

while not done:
    statet = torch.FloatTensor(state)
    probs = policynetwork(statet)
    dist = torch.distributions.Categorical(probs)
    action = dist.sample()
    logprob = dist.logprob(action)

    nextstate, reward, done, = env.step(action.item())

    logprobs.append(logprob)
    rewards.append(reward)
    state = nextstate

# 计算折扣回报
returns = []
G = 0
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)
returns = torch.FloatTensor(returns)
returns = (returns - returns.mean()) / (returns.std() + 1e-8)

# 策略梯度更新
policyloss = []
for logprob, G in zip(logprobs, returns):
    policyloss.append(-logprob * G) # 负号因为要最大化

optimizer.zero_grad()
loss = torch.stack(policyloss).sum()
loss.backward()
optimizer.step()

```

REINFORCE的缺点：方差极大，因为 G_t 可能波动很大。

减小方差的技巧：

- 使用基线 (baseline) : 用 $V(s)$ 或均值减去 G_t
- 优势函数: $A(s, a) = Q(s, a) - V(s)$
- - -

八、Actor-Critic 框架

8.1 核心思想

同时维护两个网络：

- Actor (策略网络) : 决定采取什么动作
- Critic (价值网络) : 评估动作有多好

```
class ActorCritic(nn.Module):
    def __init__(self, statedim, actiondim, hiddendim=128):
        super().__init__()
        # 共享特征提取层
        self.shared = nn.Sequential(
            nn.Linear(statedim, hiddendim),
            nn.ReLU()
        )
        # Actor: 输出动作概率
        self.actor = nn.Sequential(
            nn.Linear(hiddendim, hiddendim),
            nn.ReLU(),
            nn.Linear(hiddendim, actiondim),
            nn.Softmax(dim=-1)
        )
        # Critic: 输出状态价值
        self.critic = nn.Sequential(
            nn.Linear(hiddendim, hiddendim),
            nn.ReLU(),
            nn.Linear(hiddendim, 1)
        )
```

```
def forward(self, state):
    features = self.shared(state)
    actionprobs = self.actor(features)
    statevalue = self.critic(features)
    return actionprobs, statevalue
```

8.2 A2C / A3C

- A2C (Advantage Actor - Critic) : 同步更新, 多个环境并行收集数据
- A3C (Asynchronous A2C) : 异步更新, 每个worker独立探索
- - -

九、PPO (Proximal Policy Optimization)

9.1 核心思想

限制策略更新幅度, 避免新策略离旧策略太远导致性能崩塌。

9.2 PPO - Clip 算法

$$\mathbb{E}[\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)]$$

其中：

- $r_t(\theta) = \frac{\pi(\theta(a_t|s_t))}{\pi(\theta(a_t|s_t))}$
- $\hat{A}_t$ 是优势函数估计
- ϵ 通常取 $0.1 \sim 0.2$

直觉解释：

- 当 $r > 1 + \epsilon$ 时, clip截断, 防止策略变化过大
- 当 $r < 1 - \epsilon$ 时, clip截断, 防止策略变化过大
- 只在 "好的方向" 上限制, 不限制 "变差"

PPO 伪代码

```
def ppoupdate(observations, actions, oldlogprobs,
for epoch in range(nupdateepochs):
```

```

# 计算新的log概率和价值
newlogprobs, values = model(observations, actions)

# 概率比
ratio = torch.exp(newlogprobs - oldlogprobs)

# Clipped objective
surr1 = ratio * advantages
surr2 = torch.clamp(ratio, 1 - clipepsilon, 1 + clipepsilon)
actorloss = -torch.min(surr1, surr2).mean()

# Critic损失
criticloss = nn.MSELoss()(values.squeeze(), return_terminated_values)

# 总损失
loss = actorloss + 0.5 * criticloss - 0.01 * entropy

optimizer.zero_grad()
loss.backward()
nn.utils.clip_grad_norm(model.parameters(), max_norm)
optimizer.step()

```

9.3 GAE (广义优势估计)

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} \gamma^l \lambda^l \delta_{t+l}$$

其中 $\delta_{t+l} = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})$ 是TD误差

λ 控制偏差 - 方差权衡：

- $\lambda = 0$ ：低方差高偏差（只用一步TD误差）
- $\lambda = 1$ ：高方差低偏差（用完整轨迹，类似蒙特卡洛）
- 常用 $\lambda = 0.95$

- - -

十、SAC (Soft Actor-Critic)

10.1 核心思想

在目标函数中加入策略的信息熵，鼓励探索：

$$J(\pi) = \sum_{t=0}^T \mathbb{E}[(r_t + \gamma V_t(\pi) - V_t(\pi_{\text{old}}))] + \alpha H(\pi)$$

- α ：温度参数，控制探索程度
- $H(\pi)$ ：策略的熵，越大表示越随机

SAC的三大创新：

1. 最大熵框架：不只追求高奖励，还追求策略的随机性（鼓励探索）
2. 双Q网络：取两个Q网络的最小值，减少Q值过高估计
3. 自动温度调节： α 自适应调整，不需要手动调参

直觉：SAC的策略像水——在不确定性高的地方分散探索，在确定性高的地方集中利用。与PPO相比，SAC策略具有多样性，不容易陷入局部最优。

10.2 SAC 完整实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.distributions import Normal

class SACActor(nn.Module):
    """SAC策略网络：输出高斯分布的均值和标准差"""
    def __init__(self, statedim, actiondim, hiddendim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(statedim, hiddendim), nn.ReLU(),
            nn.Linear(hiddendim, hiddendim), nn.ReLU()
        )
        self.meanlayer = nn.Linear(hiddendim, actiondim)
        self.logstdlayer = nn.Linear(hiddendim, actiondim)
        self.logstdmin = logstdmin
        self.logstdmax = logstdmax
```

```

def forward(self, state):
    x = self.net(state)
    mean = self.meanlayer(x)
    logstd = self.logstdlayer(x)
    logstd = torch.clamp(logstd, self.logstdmin, self.logstdmax)
    return mean, logstd

def sample(self, state, epsilon=1e-6):
    """重参数化采样"""
    mean, logstd = self.forward(state)
    std = logstd.exp()
    normal = Normal(mean, std)

    # 重参数化技巧:  $z = \mu + \sigma \epsilon$ , 其中  $\epsilon \sim N(0, 1)$ 
    z = normal.rsample()
    action = torch.tanh(z) # 压缩到 [-1, 1]

    # 计算log概率(修正tanh变换)
    logprob = normal.logprob(z) - torch.log(1 - action**2)
    logprob = logprob.sum(1, keepdim=True)

    return action, logprob

class SACCrritic(nn.Module):
    """SAC双Q网络"""
    def __init__(self, statedim, actiondim, hiddendim=256):
        super().__init__()
        # Q1
        self.q1 = nn.Sequential(
            nn.Linear(statedim + actiondim, hiddendim), nn.ReLU(),
            nn.Linear(hiddendim, hiddendim), nn.ReLU(),
            nn.Linear(hiddendim, 1)
        )
        # Q2
        self.q2 = nn.Sequential(

```

```

nn.Linear(statedim + actiondim, hiddendim), nn.ReLU(),
nn.Linear(hiddendim, hiddendim), nn.ReLU(),
nn.Linear(hiddendim, 1)
)

def forward(self, state, action):
x = torch.cat([state, action], dim=1)
return self.q1(x), self.q2(x)

class SACAgent:
def init(self, statedim, actiondim, lr=3e-4, gamma=0.99):
self.gamma = gamma
self.tau = tau # 软更新系数

self.actor = SACActor(statedim, actiondim)
self.critic = SACCritic(statedim, actiondim)
self.critic_target = SACCritic(statedim, actiondim)
self.critic_target.load_state_dict(self.critic.state_dict())

self.actor_optimizer = torch.optim.Adam(self.actor.parameters())
self.critic_optimizer = torch.optim.Adam(self.critic.parameters())

# 自动温度调节
self.logalpha = torch.zeros(1, requires_grad=True)
self.alpha_optimizer = torch.optim.Adam([self.logalpha])
self.target_entropy = -actiondim # 目标熵 = -dim(A)

def update(self, batch):
states, actions, rewards, nextstates, dones = batch
states = torch.FloatTensor(states)
actions = torch.FloatTensor(actions)
rewards = torch.FloatTensor(rewards).unsqueeze(1)
nextstates = torch.FloatTensor(nextstates)
dones = torch.FloatTensor(dones).unsqueeze(1)

alpha = self.logalpha.exp()

```

```

# - - - Critic更新 - - -
with torch.no_grad():
    nextactions, nextlogprobs = self.actor.sample(nextstates, nextactions)
    q1next, q2next = self.critictarget(nextstates, nextactions)
    qnext = torch.min(q1next, q2next) - alpha nextlogprobs
    qtarget = rewards + self.gamma (1 - dones) qnext

q1, q2 = self.critic(states, actions)
criticloss = F.mse_loss(q1, qtarget) + F.mse_loss(q2, qtarget)

self.criticoptimizer.zero_grad()
criticloss.backward()
self.criticoptimizer.step()

# - - - Actor更新 - - -
newactions, logprobs = self.actor.sample(states, actions)
q1new, q2new = self.critic(states, newactions)
qnew = torch.min(q1new, q2new)

actorloss = (alpha logprobs - qnew).mean()

self.actoroptimizer.zero_grad()
actorloss.backward()
self.actoroptimizer.step()

# - - - Alpha自动调节 - - -
alphaloss = - (self.logalpha (logprobs + self.targetq))

self.alphaoptimizer.zero_grad()
alphaloss.backward()
self.alphaoptimizer.step()

# - - - 软更新目标网络 - - -
for param, targetparam in zip(self.critic.parameters(), self.critictarget.parameters()):
    targetparam.data.copy_(self.tau param.data + (1 - self.tau) targetparam.data)

```

```
return criticloss.item(), actorloss.item()
```

10.3 SAC vs PPO 选择指南

维度 PPO SAC

动作空间 离散 + 连续 连续 (离散需修改)

样本效率 中等 较高 (off-policy)

训练稳定性 好 好

探索能力 依赖熵正则化 天然最大熵

超参数敏感度 中等 较低

推荐场景 通用、RLHF 机器人控制、连续控制

实战建议：

- 连续控制 (机器人、自动驾驶) → 首选 SAC
- 离散动作 (游戏 AI) → 首选 PPO / DQN
- 大模型对齐 (RLHF) → 首选 PPO
- 不确定 → 先试 PPO (更通用)，不满意再试 SAC

- - -

十一、RLHF：从强化学习到 ChatGPT

11.1 RLHF 三步流程

Step 1：监督微调 (SFT)

用人工标注的指令 - 回答对微调基础模型

目标：学会按指令回答

Step 2：训练奖励模型

人类对多条回答排序 → 训练一个 "评委" 模型

目标：学会区分好回答和差回答

Step 3：强化学习优化 (PPO)

用奖励模型作为环境，模型通过 PPO 优化生成策略

目标：生成奖励模型评分最高的回答

11.2 奖励模型训练

```
class RewardModel(nn.Module):  
    """
```

奖励模型：输入（问题， 回答），输出标量分数

通常在SFT模型基础上，去掉最后的lmhead，加一个线性层

```
    """
```

```
    def init(self, basemodel, hidsize):  
        super().init()  
        self.basemodel = basemodel # 预训练语言模型  
        self.valuehead = nn.Linear(hidsize, 1) # 输出标量奖励  
  
    def forward(self, input_ids, attentionmask):  
        # 获取最后一层隐藏状态  
        outputs = self.basemodel(input_ids=input_ids, attentionmask=attentionmask)  
        lasthidden = outputs.lasthiddenstate # (batch, seq_len, hidsize)  
  
        # 取最后一个token的表示作为奖励  
        # 也可以用mean pooling  
        rewards = self.valuehead(lasthidden[:, -1, :]) # (batch, 1)  
        return rewards.squeeze(-1)
```

```
def computerewardloss(rewardmodel, chosenids, chosenmask, rejectedids, rejectedmask):  
    """
```

Bradley-Terry偏好模型的损失函数

核心思想：chosen的奖励应该高于rejected

```
    P(chosen > rejected) = sigmoid(rchosen - rrejected)  
    """
```

```
    chosenrewards = rewardmodel(chosenids, chosenmask)  
    rejectedrewards = rewardmodel(rejectedids, rejectedmask)
```

```
    # Bradley-Terry损失
```

```
    loss = -torch.log(torch.sigmoid(chosenrewards - rejectedrewards))  
    return loss.mean()
```

```

def trainrewardmodel(rewardmodel, dataloader, nepochs):
    """ 奖励模型训练循环 """
    optimizer = torch.optim.Adam(rewardmodel.parameters())

    for epoch in range(nepochs):
        totalloss = 0
        for batch in dataloader:
            # batch包含:chosenids, chosenmask, rejectedids, rejectedmask
            loss = computerewardloss(
                rewardmodel,
                batch['chosenids'], batch['chosenmask'],
                batch['rejectedids'], batch['rejectedmask']
            )

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm(rewardmodel.parameters())
            optimizer.step()
            totalloss += loss.item()

        print(f"Epoch {epoch+1}, Loss: {totalloss/len(dataloader)}")

```

11.3 RLHF with PPO 完整实现

```

class RLHFTrainer:
    """ RLHF训练器：用PPO优化语言模型 """
    def __init__(self, sftmodel, rewardmodel, refmodel, tokenizer,
                 lr=1e-6, klcoef=0.1, gamma=0.99, gaelambda=0.95,
                 clipepsilon=0.2, ppoepochs=4):
        self.sftmodel = sftmodel # 待优化的策略模型
        self.rewardmodel = rewardmodel # 奖励模型
        self.refmodel = refmodel # 参考模型（冻结，用于KL惩罚）
        self.tokenizer = tokenizer

        self.optimizer = torch.optim.Adam(sftmodel.parameters())

```

```

self.klcoef = klcoef # KL散度惩罚系数
self.clipped_epsilon = clipped_epsilon
self.ppo_epochs = ppo_epochs
self.gamma = gamma
self.gaelambda = gaelambda

def compute_kl_penalty(self, input_ids, attentionmask):
    """
    计算策略模型与参考模型的KL散度
    防止模型偏离参考模型太远（避免奖励黑客）
    """
    # 策略模型的log概率
    policy_logits = self.sftmodel(input_ids, attentionmask)
    policy_logprobs = F.logsoftmax(policy_logits, dim=-1)

    # 参考模型的log概率（不计算梯度）
    with torch.no_grad():
        ref_logits = self.refmodel(input_ids, attentionmask)
        ref_logprobs = F.logsoftmax(ref_logits, dim=-1)

    # KL散度： $KL(\pi_{ref} \parallel \pi_{policy})$ 
    kl = (ref_logprobs.exp() * (ref_logprobs - policy_logprobs)).sum()
    return kl.mean()

def generate_and_score(self, prompts):
    """生成回答并计算奖励"""
    # 生成回答
    responses = self.sftmodel.generate(*prompts, max_length=128)

    # 计算奖励模型分数
    rewards = self.rewardmodel(responses, attentionmask)

    # 计算KL惩罚
    kl_penalty = self.compute_kl_penalty(responses, attentionmask)

    # 总奖励 = 奖励模型分数 - KL惩罚

```

```

totalrewards = rewards - self.klcoef klpenalty

return responses, totalrewards

def ppostep(self, prompts):
    """一个PPO更新步骤"""
    # 1. 收集轨迹
    responses, rewards = self.generateandscore(prompts)

    # 2. 计算旧策略的log概率
    with torch.no_grad():
        oldlogits = self.sftmodel(responses).logits
        oldlogprobs = F.logsoftmax(oldlogits, dim=-1)

    # 3. PPO多轮更新
    for in range(self.ppoepochs):
        # 新策略的log概率
        newlogits = self.sftmodel(responses).logits
        newlogprobs = F.logsoftmax(newlogits, dim=-1)

        # 概率比
        ratio = (newlogprobs - oldlogprobs).exp()

        # 优势（简化版，实际应使用GAE）
        advantages = rewards # 简化：直接用奖励作为优势

        # PPO-Clip目标
        surr1 = ratio advantages
        surr2 = torch.clamp(ratio, 1 - self.clipepsilon,
                             1 + self.clipepsilon)
        actorloss = -torch.min(surr1, surr2).mean()

        # KL惩罚
        kl = self.computeklpenalty(responses, attentionmask)
        loss = actorloss + self.klcoef kl

    self.optimizer.zero_grad()
    loss.backward()

```

```
torch.nn.utils.clip_grad_norm(self.sftmodel.parameters(),
self.optimizer.step())

return actorloss.item(), kl.item()
```

11.4 DPO (Direct Preference Optimization)

DPO跳过奖励模型，直接从偏好数据优化策略：

$$\mathcal{L}\{\text{DPO}\}(\theta) = -\mathbb{E} \left[\log \frac{\sigma(\frac{\pi(\theta)(y_w|x)}{\pi_{\text{ref}}(y_w|x)})}{\pi_{\text{ref}}(y_w|x)} \right]$$

- y_w ：偏好回答 (chosen)
- y_l ：非偏好回答 (rejected)
- π_{ref} ：参考策略 (通常是SFT模型)
- β ：控制偏离参考策略的程度

DPO的直觉：直接增加chosen的概率，减少rejected的概率，同时不偏离参考模型太远。

```
def dpoloss(policymodel, refmodel, chosenids, chosenlogps,
rejectedids, rejectedlogps, rejectedmask, beta=0.1):
    """
```

DPO损失函数

"""

策略模型的log概率

```
chosenlogps = get_logprobs(policymodel, chosenids, chosenlogps)
rejectedlogps = get_logprobs(policymodel, rejectedids, rejectedlogps)
```

参考模型的log概率 (冻结, 不计算梯度)

```
with torch.no_grad():
    refchosenlogps = get_logprobs(refmodel, chosenids, chosenlogps)
    refrejectedlogps = get_logprobs(refmodel, rejectedids, rejectedlogps)
```

对数概率比

```
chosenlogratio = chosenlogps - refchosenlogps
rejectedlogratio = rejectedlogps - refrejectedlogps
```

```

# DPO损失
logits = beta (chosenlogratio - rejectedlogratio)
loss = -F.logsigmoid(logits).mean()

# 可选：记录指标
with torch.no_grad():
# chosen的平均奖励应该高于rejected
chosenreward = beta chosenlogratio
rejectedreward = beta rejectedlogratio
accuracy = (chosenreward > rejectedreward).float()

return loss, accuracy

def get_logprobs(model, input_ids, attentionmask):
    """ 获取模型对输入序列的log概率"""
    outputs = model(input_ids=input_ids, attentionmask=attentionmask)
    logits = outputs.logits[:, :-1, :] # 去掉最后一个位置的预测
    labels = input_ids[:, 1:] # 去掉第一个位置(shifted)

# 计算每个token的log概率
pertokenlogps = torch.gather(
    F.logsoftmax(logits, dim=-1),
    dim=2,
    index=labels.unsqueeze(2)
).squeeze(2)

# 只计算非padding位置的log概率
mask = attentionmask[:, 1:].float()
return (pertokenlogps * mask).sum(dim=1) / mask.sum(dim=1)

def train_dpo(policy_model, ref_model, dataloader, n_epochs):
    """ DPO训练循环"""
    optimizer = torch.optim.Adam(policy_model.parameters())
    ref_model.eval() # 参考模型冻结

```

```

for epoch in range(nepochs):
    totalloss = 0
    totalacc = 0

    for batch in dataloader:
        loss, acc = dpoloss(
            policymodel, refmodel,
            batch['chosenids'], batch['chosenmask'],
            batch['rejectedids'], batch['rejectedmask'],
            beta=beta
        )

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm(policymodel.parameters())
        optimizer.step()

    totalloss += loss.item()
    totalacc += acc.item()

    print(f"Epoch {epoch+1}, Loss: {totalloss/len(dataloader)}, "
          f"Accuracy: {totalacc/len(dataloader):.4f}")

```

11.5 RLHF vs DPO 对比

维度 RLHF DPO

训练流程 SFT → Reward Model → PPO SFT → DPO

需要奖励模型 需要单独训练 不需要

训练稳定性 较差（PPO调参困难） 较好（类似SFT训练）

计算成本 高（4个模型同时加载） 低（2个模型）

数据需求 偏好排序数据 偏好排序数据（相同）

效果上限 理论更高 接近RLHF

实现难度 高 低

适用场景 大规模对齐（ChatGPT） 中小规模、快速迭代

实战建议：

- 从DPO开始：实现简单，效果也不错，适合快速验证
- RLHF当需要极致效果时：PPO理论上限更高，但调参成本大
- 混合策略：先用DPO快速迭代，确定偏好数据质量后再用RLHF精调
- - -

十二、算法演进路线图

MDP（形式化框架）

- Bellman方程（理论基石）
- 动态规划（值迭代 / 策略迭代）
- Q-learning（无模型值函数方法）
- DQN（深度化 + 经验回放 + 目标网络）
- 策略梯度（直接优化策略）
- Actor-Critic（混合架构，降低方差）
- PPO（稳定可靠的策略优化）
- SAC（最大熵强化学习）
- RLHF（大模型对齐应用）
- DPO（简化版对齐）
- - -

十三、强化学习的实际应用

应用 环境 算法 说明

游戏AI Atari、围棋 DQN、MCTS+RL AlphaGo、OpenAI Five

机器人控制 MuJoCo、真实机器人 SAC、PPO 灵巧操作、导航

自动驾驶 CARLA模拟器 SAC、PPO 变道、超车策略

推荐系统 用户交互日志 Contextual Bandit 实时个性化推荐

大模型对齐 奖励模型 PPO、DPO ChatGPT的RLHF

芯片设计 EDA工具 RL+搜索 Google用RL设计TPU布局

化学分子 分子生成环境 GFlowNet 药物分子设计

13.1 Gymnasium 环境使用


```
import gymnasium as gym
```

创建环境

```
env = gym.make("CartPole-v1")
```

基础交互循环

```
observation, info = env.reset()
for step in range(1000):
    action = env.action_space.sample() # 随机策略
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

常用Gymnasium环境：

环境 类型 状态空间 动作空间 难度

CartPole-v1	经典控制	Box(4,)	Discrete(2)	入门
MountainCar-v0	经典控制	Box(2,)	Discrete(3)	中等
LunarLander-v2	经典控制	Box(8,)	Discrete(4)	中等
Acrobot-v1	经典控制	Box(6,)	Discrete(3)	中等
Ant-v4	MuJoCo	Box(111,)	Box(8,)	高级
HalfCheetah-v4	MuJoCo	Box(17,)	Box(6,)	高级
Atari/	游戏	Box(210,160,3)	Discrete(n)	高级

13.2 DQN完整训练示例 (CartPole)

```
import gymnasium as gym
import torch
import torch.nn as nn
import numpy as np
from collections import deque
import random
```

=== 环境封装 ===

```
class CartPoleEnv:
```

```
    """CartPole环境封装, 添加预处理"""
```

```
    def init(self):
```

```
        self.env = gym.make("CartPole-v1")
```

```
        self.statedim = 4
```

```
        self.actiondim = 2
```

```
    def reset(self):
```

```
        state, = self.env.reset()
```

```
        return state
```

```
    def step(self, action):
```

```
        state, reward, terminated, truncated, = self.env.
```

```
        done = terminated or truncated
```

```
        return state, reward, done
```

=== 训练循环 ===

```
def traindqnn(nepisodes=1000, maxsteps=500):
```

```
    env = CartPoleEnv()
```

```
    agent = DQNAgent(statedim=4, actiondim=2, lr=1e-3
```

```
    epsilon=1.0, epsilonmin=0.01, epsilondecay=0.995)
```

```
    scores = []
```

```
    scoreswindow = deque(maxlen=100) # 最近100个episode的
```

```
    for episode in range(nepisodes):
```

```
        state = env.reset()
```

```
        totalreward = 0
```

```
        for step in range(maxsteps):
```

```
            # 选择动作
```

```
            action = agent.act(state)
```

```
            # 执行动作
```

```
            nextstate, reward, done = env.step(action)
```

```

# 存入经验
agent.buffer.push(state, action, reward, nextstate)
# 学习
agent.update(batchsize=64)

state = nextstate
totalreward += reward

if done:
    break

scores.append(totalreward)
scoreswindow.append(totalreward)

# 打印进度
if episode % 50 == 0:
    avgscore = np.mean(scoreswindow)
    print(f"Episode {episode}, Avg Score: {avgscore:.3f}")
    print(f"Epsilon: {agent.epsilon:.3f}")

# 解决条件：连续100个episode平均分>=475
if len(scoreswindow) == 100 and np.mean(scoreswindow) >= 475:
    print(f"\n环境在Episode {episode}被解决!")
    break

return scores

```

运行训练

```
scores = traindqnn()
```

13.3 PPO完整训练示例(LunarLander)

```

class RolloutBuffer:
    """PPO的轨迹缓冲区"""
    def __init__(self):
        self.states = []
        self.actions = []

```

```

self.logprobs = []
self.rewards = []
self.dones = []
self.values = []

def push(self, state, action, logprob, reward, done):
    self.states.append(state)
    self.actions.append(action)
    self.logprobs.append(logprob)
    self.rewards.append(reward)
    self.dones.append(done)
    self.values.append(value)

def computereturns(self, gamma=0.99, gaelambda=0.99):
    """ 计算GAE优势和折扣回报 """
    rewards = np.array(self.rewards)
    values = np.array(self.values)
    dones = np.array(self.dones)

    advantages = np.zeros_like(rewards)
    lastgae = 0

    for t in reversed(range(len(rewards))):
        if t == len(rewards) - 1:
            nextvalue = 0
        else:
            nextvalue = values[t + 1]

        delta = rewards[t] + gamma * nextvalue - (1 - dones[t]) * values[t]
        lastgae = delta + gamma * gaelambda * lastgae
        advantages[t] = lastgae

    returns = advantages + values
    return advantages, returns

def clear(self):

```

```

self.states.clear()
self.actions.clear()
self.logprobs.clear()
self.rewards.clear()
self.dones.clear()
self.values.clear()

def trainppo(nepisodes=2000, maxsteps=1000, updateinterval=1000):
    """PPO完整训练循环"""
    env = gym.make("LunarLander-v2")
    model = ActorCritic(statedim=8, actiondim=4)
    optimizer = torch.optim.Adam(model.parameters())
    buffer = RolloutBuffer()

    for episode in range(nepisodes):
        state, _ = env.reset()
        totalreward = 0

        for step in range(maxsteps):
            statet = torch.FloatTensor(state)
            actionprobs, value = model(statet)
            dist = torch.distributions.Categorical(actionprobs)
            action = dist.sample()
            logprob = dist.logprob(action)

            nextstate, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            buffer.push(state, action.item(), logprob.item(), value.item(), done)
            state = nextstate
            totalreward += reward

        # 收集够数据后更新
        if len(buffer.states) >= updateinterval:
            advantages, returns = buffer.computereturns()

```

```

# 转为张量
statest = torch.FloatTensor(np.array(buffer.statest))
actionst = torch.LongTensor(buffer.actions)
oldlogprobst = torch.FloatTensor(buffer.logprobs)
advantagest = torch.FloatTensor(advantages)
returnst = torch.FloatTensor(returns)

# 标准化优势
advantagest = (advantagest - advantagest.mean())

# PPO更新
for i in range(10): # 10个epoch
    newprobs, newvalues = model(statest)
    dist = torch.distributions.Categorical(newprobs)
    newlogprobs = dist.logprob(actionst)
    entropy = dist.entropy().mean()

    ratio = (newlogprobs - oldlogprobst).exp()
    surr1 = ratio * advantagest
    surr2 = torch.clamp(ratio, 0.8, 1.2) * advantagest
    actorloss = -torch.min(surr1, surr2).mean()

    criticloss = nn.MSELoss()(newvalues.squeeze(), returns)
    loss = actorloss + 0.5 * criticloss - 0.01 * entropy

    optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm(model.parameters(), 0.5)
    optimizer.step()

    buffer.clear()

    if done:
        break

    if episode % 50 == 0:

```

```
print(f"Episode {episode}, Reward: {total_reward}")

env.close()

- - -
```

十四、现实挑战与未来方向

当前难点

挑战	说明	可能的解决方案
奖励设计	设计合适的奖励函数极困难	逆强化学习、RLHF
样本效率	需要海量交互数据	基于模型的RL、离线RL
训练稳定性	训练过程不稳定	PPO、SAC
泛化性	在新环境中表现差	元学习、域随机化
安全约束	探索可能产生危险动作	约束RL、安全RL

未来方向

方向 说明

- 基于模型的RL 学习环境模型，提升样本效率
- 离线RL 从历史数据中学习，不需要在线交互
- 多智能体RL 多个智能体协作或竞争
- 世界模型 融合感知、记忆与推理
- LLM + RL 语言模型作为策略，RL优化决策
- 层次RL 拆分复杂任务为子目标

- - -

核心总结：强化学习是连接 " 理解 " 与 " 行动 " 的关键桥梁。从MDP到Q-learning到DQN到PPO，一步步 " 让机器学会做决策 "。PPO是当前最实用的算法，RLHF / DPO是大模型对齐的关键技术。强化学习的核心在于探索 - 利用困境和样本效率。